

Compte Rendu : TD 3 - Accès aux données avec JDBC

Délio Duchesne-Brajon

[Compte Rendu : TD 3 - Accès aux données avec JDBC](#)

[Introduction](#)

[1. Gestion des listes de livres](#)

[2. Extension des données](#)

[3. Affichage et vérification](#)

[4. Création de l'interface utilisateur](#)

[5. Attribut privé de la classe Accueil](#)

[6. Initialisation automatique au lancement](#)

[7. Design final de la fenêtre](#)

Introduction

Ce compte rendu détaille les étapes de réalisation d'une application de gestion de livres. Le projet articule la programmation orientée objet (POO), la gestion de listes dynamiques, la création d'une interface graphique (JavaFX/Swing) et la connexion à une base de données MySQL.

1. Gestion des listes de livres

L'objectif initial était de créer une structure capable de stocker nos objets en mémoire durant l'exécution du programme.

- **Attentes** : Déclarer une `ArrayList` de type `Livre` et y ajouter manuellement les deux premiers ouvrages.

- **Pourquoi est-ce important ?** En programmation, les données doivent être regroupées pour être manipulées efficacement. Utiliser une liste plutôt que des variables isolées permet de traiter 2 ou 2000 livres avec le même code.

```
public static void main(String[] args) {
    ArrayList<Livre> livres = new ArrayList<>();
}
```

2. Extension des données

Nous avons ensuite enrichi notre collection en ajoutant deux ouvrages supplémentaires.

- **Attentes** : Valider la capacité de la liste à s'agrandir dynamiquement.
- **Pourquoi est-ce important ?** Une application réelle doit être évolutive. Savoir ajouter des éléments à une liste existante est la base de toute gestion de stock.

```
// Exemple de création de livres
Livres livres = new ArrayList<>();
Livres.add(new Livre("titre 1", auteur: "auteur 1", prix: 1));
Livres.add(new Livre("titre 2", auteur: "auteur 2", prix: 2));
Livres.add(new Livre("titre 3", auteur: "auteur 3", prix: 3));
Livres.add(new Livre("titre 4", auteur: "auteur 4", prix: 4));
Livres.forEach(Livres::Afficher);
```

3. Affichage et vérification

Pour s'assurer que les données sont correctement stockées, une méthode d'affichage a été utilisée pour parcourir la liste.

- **Attentes** : Utiliser une boucle pour appeler la méthode `Afficher()` de chaque objet `Livre`.
- **Pourquoi est-ce important ?** La visibilité est essentielle pour le débogage. Si on ne peut pas afficher ce qui est en mémoire, on ne peut pas savoir si le programme fonctionne réellement.

```

for (Livre livre : livres) {
    livre.Afficher();
    System.out.println(x: "");
}

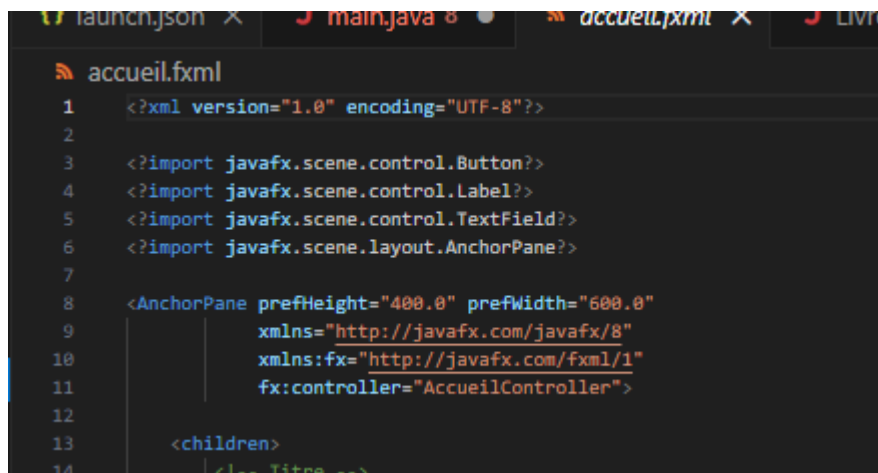
launch, javafx(args);

```

4. Création de l'interface utilisateur

Le passage à une "Application Windows" nommée **Accueil** marque le début de la partie visuelle du projet.

- **Attentes** : Configurer l'environnement de travail et générer la fenêtre principale via un outil de design (WindowBuilder ou JavaFX).
- **Pourquoi est-ce important ?** L'interface est le pont entre l'utilisateur et le code. Un programme sans fenêtre est inutilisable pour le grand public.



```

1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import javafx.scene.control.Button?>
4  <?import javafx.scene.control.Label?>
5  <?import javafx.scene.control.TextField?>
6  <?import javafx.scene.layout.AnchorPane?>
7
8  <AnchorPane prefHeight="400.0" prefWidth="600.0"
9      xmlns="http://javafx.com/javafx/8"
10     xmlns:fx="http://javafx.com/fxml/1"
11     fx:controller="AccueilController">
12
13     <children>
14         <!-- Titre -->

```

5. Attribut privé de la classe Accueil

Au sein de la classe Accueil, un attribut privé a été créé pour gérer la bibliothèque de manière sécurisée.

- **Attentes** : Déclarer `private ArrayList<Livre> al = new ArrayList<Livre>();`.

- **Pourquoi est-ce important ?** Le mot-clé `private` assure l'encapsulation. Cela garantit que la liste des livres ne peut pas être modifiée par erreur depuis une autre partie du programme.

```
J Accueil.java > ...  
  
1  import java.io.File;  
2  import java.net.URL;  
3  import java.util.ArrayList;  
4  import javafx.application.Application;  
5  import javafx.fxml.FXMLLoader;  
6  import javafx.scene.Parent;  
7  import javafx.scene.Scene;  
8  import javafx.stage.Stage;  
9  
10 public class Accueil extends Application {  
11  
12     private ArrayList<Livre> al = new ArrayList<Livre>();  
13
```

6. Initialisation automatique au lancement

Pour que l'application soit opérationnelle dès l'ouverture, les 4 livres ont été instanciés directement dans le constructeur.

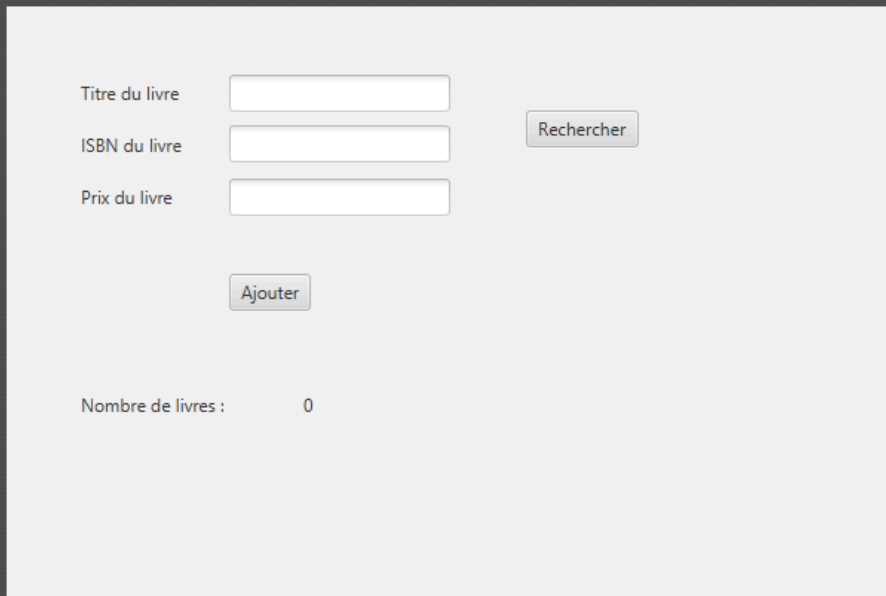
- **Attentes** : Placer le code de création des objets avant l'appel de la méthode `initialize()`.
- **Pourquoi est-ce important ?** Cela définit l'état initial du logiciel. Un utilisateur ne doit pas se retrouver face à une interface vide s'il y a déjà des données existantes.

```
Void.Livre initialiseLivres(){  
    return new Void.Livre(ISBN, titre, auteur, prix);  
}
```

7. Design final de la fenêtre

La dernière étape a consisté à réaliser l'interface visuelle comprenant les champs de saisie (ISBN, Titre, Prix) et les boutons d'action.

- **Attentes** : Mettre en place les labels, les zones de texte et les boutons "Rechercher" ou "Ajouter".
- **Pourquoi est-ce important ?** Une bonne ergonomie permet une saisie rapide et sans erreur des données.



The image shows a light gray rectangular form on a dark gray background. Inside the form, there are three labels on the left: "Titre du livre", "ISBN du livre", and "Prix du livre". Each label is followed by a white rectangular input field. To the right of these input fields is a button labeled "Rechercher". Below the input fields is another button labeled "Ajouter". At the bottom left of the form, there is a label "Nombre de livres :" followed by the number "0".